

ECE 408
GPT2 Milestone 3 Report

Maximum Warp Warriors
Arjun Vedantham, Neil Ghose, Pilar Luiz, Alex Jung

Fall 2025

Introduction

This report details the optimizations implemented in Milestone 3, building upon the foundation established in previous milestones. In Milestone 1, we implemented the baseline, non-optimized GPT-2 forward pass kernels. In Milestone 2, we implemented several required optimizations including register tiling for matrix multiplication (req_0), tensor core utilization (req_1), cuBLAS integration (req_2), and reduction optimizations (req_3), which significantly improved performance from the baseline.

For Milestone 3, we implemented both required optimizations and proposed optimizations from our Milestone 2 report. The required optimizations include Flash Attention (req_4), configuration sweep and optimization (req_5), constant memory utilization (req_6), the `__restrict__` keyword (req_7), local/windowed attention (req_8), and Split-K matrix multiplication (req_9). Additionally, we implemented two proposed optimizations: a Key-Value (KV) cache for autoregressive generation and NVIDIA CUTLASS integration for matrix multiplication operations.

Now that we have optimizations from the last milestone, we can perform more meaningful performance comparisons than were possible in Milestone 2. This enables us to assess whether new optimizations provide additional performance benefits beyond what we've already achieved, and come to understand how different optimizations interact and complement each other. Additionally, Milestone 2 optimizations mainly targeted matrix multiplication kernels, but Milestone 3 provides an opportunity to focus on improving attention.

The following sections detail the implementation, performance analysis, and insights gained from each optimization, with comparisons made against both baseline implementations from Milestone 1 and optimized implementations from Milestone 2 where applicable.

Required Performance Optimizations

Flash Attention (req_4)

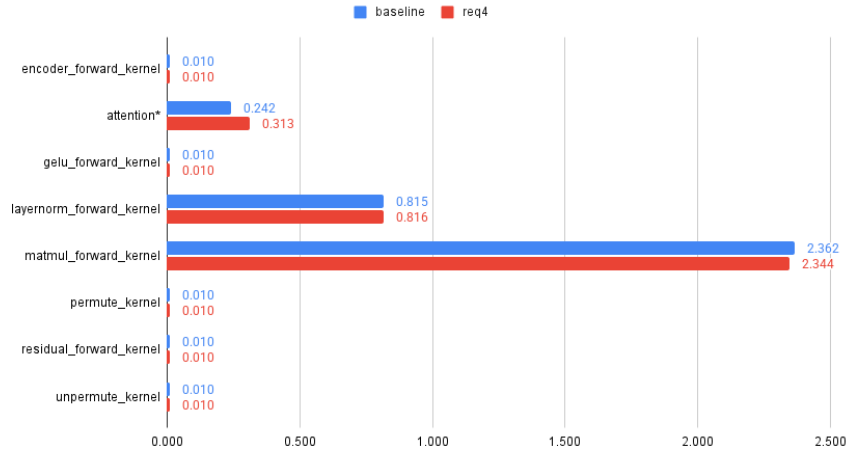
For this optimization, we replaced the standard multi-kernel attention mechanism with a Flash Attention implementation (specifically following Algorithm 1 from the FlashAttention paper).

We implemented a fused CUDA kernel `flash_attn_kernel_tiled` in `attention`. We used SRAM Tiling to reduce High Bandwidth Memory accesses. We split the Query Q , Key K , and Value V matrices into blocks. Q is loaded into thread-local registers (simulating the inner loop). K and V are loaded into Shared Memory (SRAM) in chunks defined by `TILE_SIZE` (set to 32). The kernel computes the attention scores $S = QK^T$, applies the casual mask, and computes the weighted sum $O = \text{softmax}(S)V$ in a single pass without ever materializing the large $N \times N$ attention matrix to global memory. To handle the softmax normalization without seeing the full row at once, we implemented Online Softmax. We maintained running statistics for the maximum score m and the sum of exponentials l to rescale the output incrementally as we processed new tiles.

The primary motivation was to address the memory bandwidth bottleneck. Standard attention has a memory complexity of $O(N^2)$ because it must write the attention score matrix S and probability matrix P to HBM. By fusing these operations and keeping intermediate results in SRAM, Flash Attention reduces HBM accesses to linear $O(N)$ with respect to sequence length.

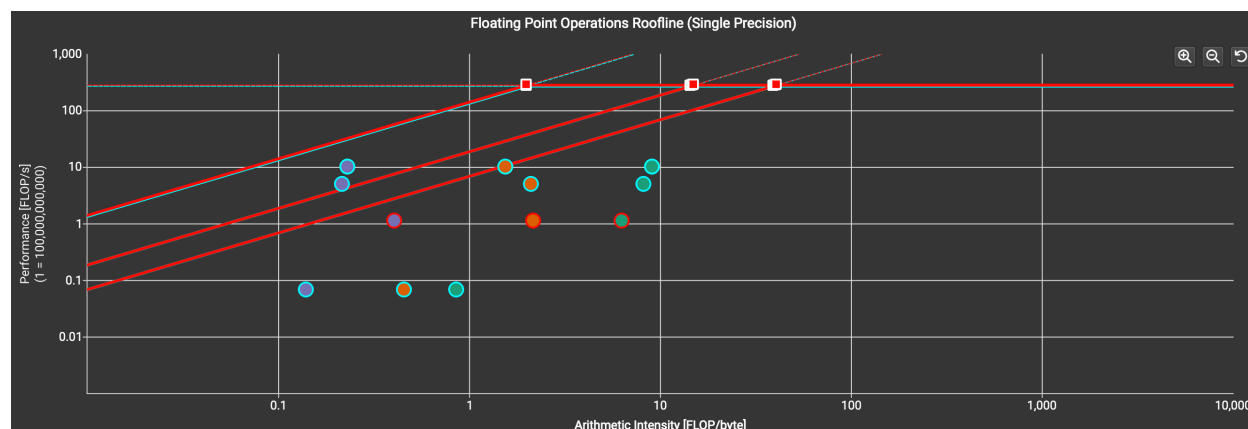
Contrary to the theoretical asymptotic speedup, the Flash Attention kernel was slightly slower than the baseline in this specific test environment. The test harness utilizes a short sequence length $N = 128$. Flash Attention is an asymptotic optimization where its benefits dominate when N is large. At small N , the overhead of initializing tiles, managing the online softmax loop, and synchronizing threads outweighs the savings gained from reduced HBM. The baseline `flattened_matmul` achieved 73% Memory Throughput, fully saturating the hardware. The `flash_attn_kernel_tiled` achieved only 8.8% Memory Throughput and 5.2% Compute Throughput. The optimized kernel is Latency Bound. The GPU threads are spending the majority of their time waiting rather than moving data. This is expected behavior for Flash Attention on small sequences.

Average Durations [ms] of Baseline vs. Flash Attention



* For the baseline, the bar labeled attention represents the combined aggregate runtime of the three distinct kernels that Flash Attention replaces: `flattened_matmult`, `softmax_forward_kernel`, `flattened_matmul`. This aggregation allows for a direct comparison against the single fused `flash_attn_kernel_tiled` kernel in the Optimized version.

Despite the runtime deficit on short sequences, Nsight Systems provided concrete evidence of successful architectural optimization: the total kernel launch count per inference pass dropped from 171 (Baseline) to 147 (Flash Attention). This reduction corresponds exactly to the fusion of 3 attention kernels per layer across the 12-layer model.



The roofline model allows us to determine if a kernel is compute bound or memory bound. The top two kernels, `flattened_matmul_kernel` and `flattened_matmult_kernel` from baseline model, are achieving the highest GFLOPS, putting them closest to the theoretical peak performance.

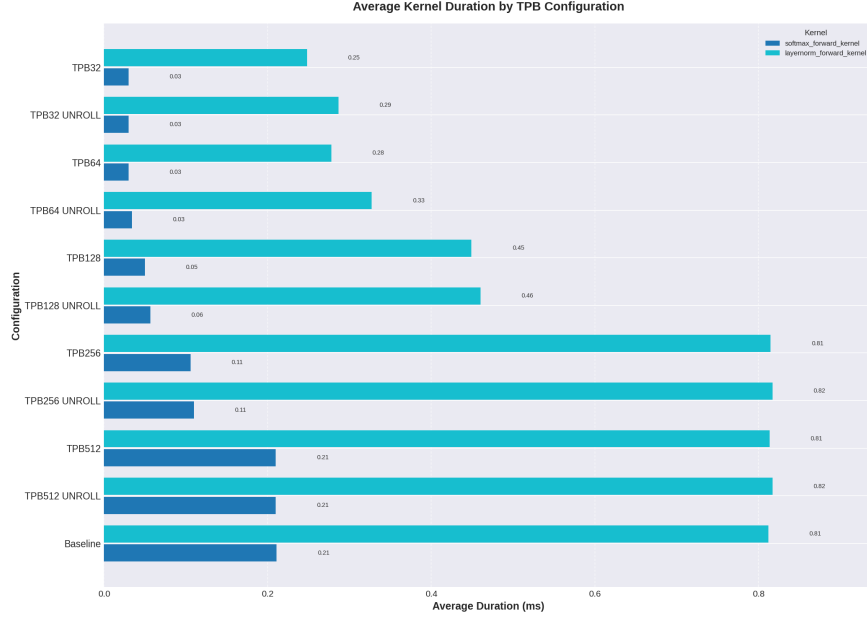
The next only red kernel `flash_attn_kernel_tiled` sits significantly lower on the Y-axis compared to the baseline matmuls. Crucially, it resides far below the roof. It is not touching the slanted line (Memory Bound) because its Memory Throughput is only 8.8%. Also, it is not touching the flat line (Compute Bound) because its Compute Throughput is only 5.2%. The floating position indicates the kernel is Latency Bound. The GPU threads are stalled, waiting for synchronization barriers (`__syncthreads()`) and instruction overhead associated with the tiling logic and online softmax loop. It is not effectively utilizing the available bandwidth or compute capacity at this short sequence length.

`softmax_forward_kernel` is located at the lowest on the chart. Softmax has very low Arithmetic Intensity, meaning it does very little math per byte loaded. It is inherently memory-limited, but its extremely low position suggests it is also suffering from inefficient memory access patterns (latency), limiting its throughput to just 4.8%.

Configuration Sweep/Optimization (req_5)

In order to find optimal parameters for running the model, we wrote a script to test different parameters. While Kernel specific parameters are swept in their specific section (see req_9, req_8), this section is dedicated to overall architecture parameters. Notably, we tested the number of threads per block the use of loop unrolling in each kernel. Additionally, we tested the model-specific hyperparameters of batch size and sequence length.

We achieved this by modifying our kernels so that the variables can be defined in the Makefile, and then wrote scripts to build different versions of the kernel depending on what we wanted to sweep over. Finally, we would launch all the kernels in a single profiling slurm job.



For most kernels (not shown) there is little to no change in the duration of the kernels. However, for the softmax and layernorm kernels, we see that loop unrolling and more threads per block actually hurts performance. Unrolling seems to make the smallest difference, likely because by default the compiler aggressively unrolls, suggesting that the overhead of the small additional extra unrolling done by this optimization is worse than the speed up from the extra unrolls.

Having fewer threads per block is likely only worse in these kernels because this was run on the baseline implementation, which do not benefit from more parallelism. Thus, the overhead of launching more threads per block likely hurts performance.

Table 1: Total Duration (ms) by B and T

B \ T	32	64	128	256
2	1.05	1.93	3.71	6.77
4	1.80	3.50	6.17	—
8	3.33	5.83	—	—
16	5.66	—	—	—

Varying the batch size and sequence length shows that generally having smaller values will allow for faster processing times. However, this results in a lower achieved occupancy ratio. Having larger batch sizes and sequence lengths leads to greater saturation of the SMs in the GPU, with higher achieved occupancy at the cost of higher per-kernel latency. Generally, it seems that the slowdown incurred by increasing these values (B , T) is less than what it would be to launch them with smaller parameters that could lead to greater serialization when scheduling warps.

Table 2: Achieved Occupancy for $Q * K^T$ across selected batch size/sequence length configurations

B	T	Achieved Occupancy (%)
2	32	32.15%
8	64	87.34%
2	256	94.17%
4	64	78.09%

Constant Memory (req_6)

The primary motivation for implementing Constant Memory optimization was to reduce global memory bandwidth pressure for frequently accessed, read-only parameters. CUDA Constant Memory utilizes a dedicated cache that is optimized for broadcasting, where a single memory read can be served to all threads in a warp simultaneously if they access the same address.

We selected Layer Normalization as the sole candidate for this optimization based on the following criteria:

1. Memory constraints: The available constant memory is hard-limited.
2. Access patterns: Constant memory performance degrades significantly if threads in a warp access different addresses (serialization).

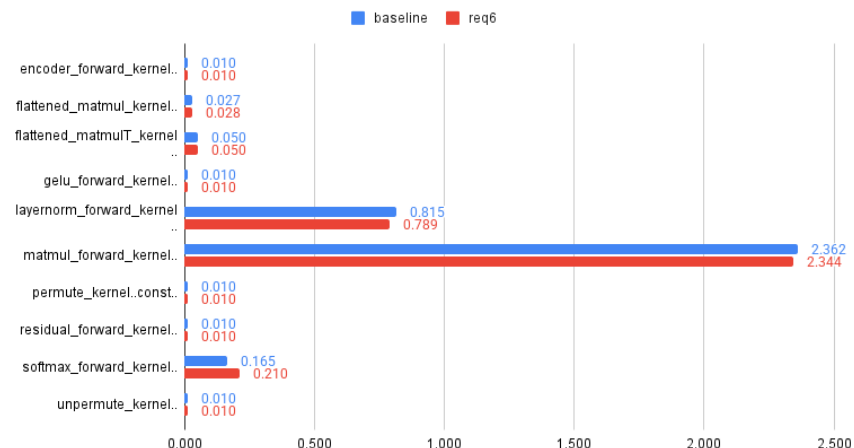
The implementation involved three key changes to the `layernorm` module. We defined fixed-size global arrays weight and bias with the `__constant__` qualifier. We replaced standard `cudaMemcpy` with `cudaMemcpyToSymbol` in the host function to transfer the model weights into the device’s constant memory bank prior to kernel launch. We removed the pointer arguments for weights and biases from the kernel signature. The kernel logic was updated to read directly from the global constant symbols.

The profiling results demonstrate a measurable improvement in the Layer Normalization kernel. The `layernorm_forward_kernel` duration decreased from 20.37 ms to 19.72 ms. This represents a 3.2% speedup for this specific kernel. The improvement is attributed to the reduction in global memory transactions. By utilizing the constant cache’s broadcast capability, the GPU reduced the bandwidth required to load the parameters, freeing up L2 cache bandwidth for the input and output tensors.

Despite the success at the kernel level, the overall impact on the model’s inference speed was marginal. The total inference time decreased from 139.66 ms to 138.63 ms ($\approx 0.74\%$ improvement). The profiling data reveals that Matrix Multiplication consumes approximately 82% of the total execution time. Layer Normalization, while optimized, accounts for only 14% of the runtime. Therefore, even a significant speedup in `layernorm` yields only less than a percent improvement in total Tokens Per Second.

Average Kernel Durations Comparison: Baseline vs. Constant Memory

Average Durations [ms] of Baseline vs. Constant Memory Opt



An interesting observation was the apparent regression in the `softmax_forward_kernel` (0.165 ms to 0.210 ms) and the slight shift in `matmul`. Since the code for these kernels was untouched, this variance provides insight into the noise floor of GPU profiling.

`__restrict__` (req-7)

The `__restrict__` keyword is a CUDA compiler hint that informs the compiler that pointer parameters do not alias with each other, meaning they point to non-overlapping memory regions. This allows the compiler to perform more aggressive optimizations, particularly around register allocation, instruction scheduling, and memory access patterns. Compilers typically assume pointers alias, leading to inefficient code due to added redundancy for correctness. The `__restrict__` keyword acts as a promise to the compiler that any data written through that pointer will not be read by any other pointer also marked with `__restrict__`. This enables the compiler to perform aggressive optimizations on pointer usage, potentially leading to performance improvements in specific scenarios.

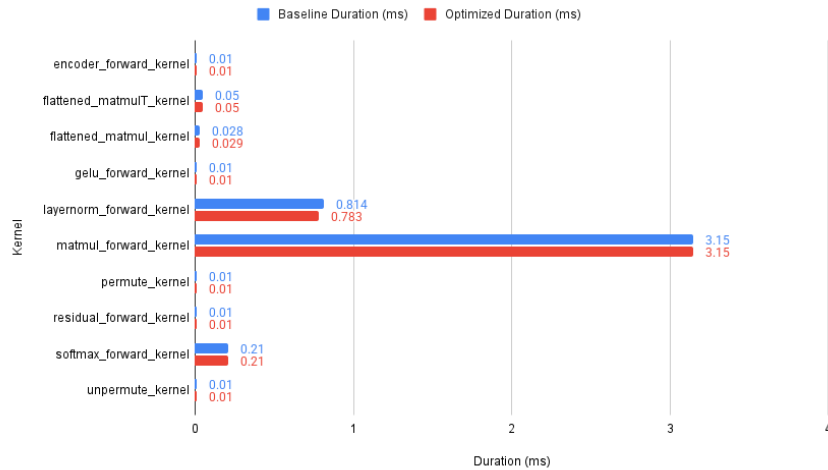
For our implementation, we applied the `__restrict__` qualifier to all pointer arguments across our CUDA kernel implementations because none of the pointers alias each other. This enables the compiler to safely reorder memory operations and eliminate redundant global memory accesses.

Table 3: Memory Performance Comparison: Baseline vs. `__restrict__` (Per Kernel)

Kernel	Bandwidth (%)		Throughput (GB/s)	
	Baseline	<code>__restrict__</code>	Baseline	<code>__restrict__</code>
encoder_forward	20.78	20.08	143.02	138.53
layernorm_forward	1.01	1.01	2.36	2.43
matmul_forward	31.74	31.77	56.15	56.30
permute	59.64	62.99	412.12	436.09
flattened_matmulT	29.20	29.18	57.87	58.50
softmax_forward	5.28	5.18	7.77	8.08
flattened_matmul	60.98	61.53	108.87	105.43
unpermute	26.01	25.99	179.64	178.77
residual_forward	47.45	46.61	327.11	321.94

Graph 1: Average Kernel Durations Comparison: Baseline vs. `__restrict__`

Restrict Optimization: Baseline vs Optimized Kernel Durations (ms)



The `__restrict__` keyword is beneficial in kernels where multiple input buffers are accessed simultaneously, such as in matrix multiplication operations where the compiler can now safely assume that reading from input and weight arrays will not interfere with writes to output arrays, enabling better register allocation and instruction-level parallelism. It also helps in kernels with loop-carried dependencies, such as the reduction loops in matmul operations where each thread accumulates a sum across multiple iterations, or sequential processing in softmax, where `__restrict__` allows the compiler to reorder memory operations and pipeline loads

and stores more aggressively. Additionally, in attention kernels like `permute_kernel` and `unpermute_kernel` that perform non-contiguous memory accesses, `__restrict__` helps the compiler understand that source and destination buffers are independent, allowing for better prefetching and cache utilization.

When examining memory bandwidth and throughput metrics, the `permute` kernel shows the most significant improvement, with bandwidth utilization increasing by 5.6% and throughput by 5.8%. However, profiling kernel execution durations reveals a different pattern, where most kernels show minimal changes within 0.001ms of the baseline, with the exception of `layernorm_forward_kernel` which decreases from 0.814ms to 0.783ms. This kernel benefits from the optimization because it performs complex non-contiguous memory accesses, reading from a concatenated QKV input buffer and writing to three separate output buffers for query, key, and value matrices. The compiler’s ability to reorder these independent memory operations is particularly valuable here, as it can pipeline loads from the input buffer with stores to the three output arrays more aggressively, better utilize memory coalescing by understanding that writes to different output arrays do not interfere, and eliminate redundant address calculations and memory barriers.

Most other kernels show minimal impact because they access each input element only once per thread, eliminating opportunities for register caching. Kernels like matrix multiplication, encoder, and element-wise operations (GELU, residual, unpermute) perform single-pass operations where each thread reads input elements once and writes output elements once. Without repeated data access patterns, `__restrict__` provides limited benefit beyond what the compiler can already infer from the code structure.

Several kernels demonstrate that `__restrict__` can introduce performance regressions, highlighting that compiler optimizations are not universally beneficial. The `encoder_forward` kernel shows a 3.1% throughput decrease (143.02 to 138.53 GB/s) and bandwidth reduction (20.78% to 20.08%), while `residual_forward` shows a 1.6% throughput decrease (327.11 to 321.94 GB/s) and bandwidth reduction (47.45% to 46.61%). The `flattened_matmul` kernel exhibits a 3.2% throughput decrease (108.87 to 105.43 GB/s) despite a slight bandwidth increase, and the `flattened_matmul_kernel` shows a slight duration increase (0.028ms to 0.029ms) in execution time measurements. These throughput regressions can occur because `__restrict__` enables compiler optimizations that introduce overhead without corresponding benefits for simple kernels, such as register spilling which can reduce occupancy.

The results demonstrate that `__restrict__` is most effective when kernels exhibit specific access patterns: either complex multi-buffer memory operations that benefit from improved memory scheduling (as seen in `permute`’s bandwidth gains) or multi-pass data access that enables register caching (as seen in `layernorm`’s duration gains). For single-pass kernels with straightforward memory patterns, the optimization provides minimal or negative benefit.

Local/Windowed Attention (req.8)

We adapted the given CPU implementation of windowed attention for the GPU, unrolling the loops in the CPU version to different dimensions of the block and grid configurations. Originally, we had difficulty getting this to work properly when parallelized over the sequence length, but eventually we were able to fix this issue and use a single large global memory buffer to hold each of the windows for each thread. Later, this buffer was converted into shared memory arrays per block, and passed as a launch parameter to the kernel. We also leveraged kernel fusion, combining the $V_o = Q * K^T$ matrix multiplication, softmax, and $V_o * V$ multiplication into a single kernel. This was more convenient than using three different kernels as in our baseline, since the softmax would have to be computed over only the windowed set of token values.

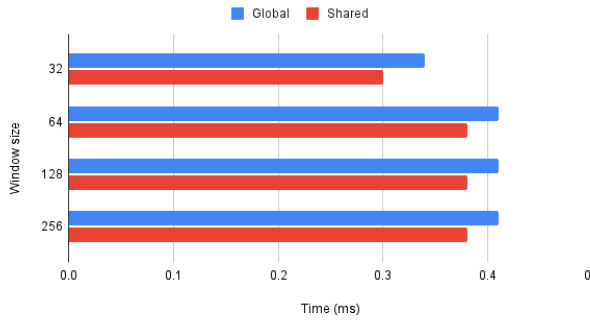
Our baseline kernel implementation took 0.29 ms altogether (running the $Q * K^T$, softmax, and $product * V$ operations). We profiled across a range of window sizes, going from a 32 token window to a 256 token window. The maximum sequence length for GPT-2 is 1024 tokens, but our evaluations assumed a maximum width of 64 - as such, we did not anticipate a significant speedup for the higher width windows since it would be equivalent to processing the entire sequence length. As the M3 README explained, lower window sizes (e.g. 32) resulted in the model forgetting the context - we found that when running the token generation job, the model’s response would only be relevant to the prompt for only a few words before veering off into

another topic.

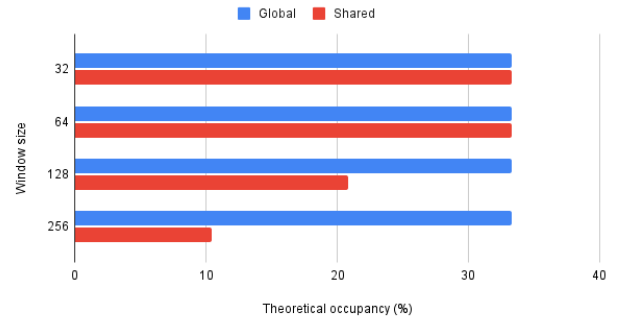
Table 4

Memory Type	Window Size	Matmul and Softmax (combined) Duration
Global	32	0.34
	64	0.41
	128	0.41
	256	0.41
Shared	32	0.30
	64	0.38
	128	0.38
	256	0.38

Duration (global vs shared memory)



Theoretical occupancy (global vs shared memory)



(a) Duration of each windowed attention kernel (same as table)

(b) Theoretical occupancy of each kernel configuration

For the global memory implementation, the achieved occupancy was slightly higher for the lower window sizes. This lines up with the memory throughput patterns - the lowest window size had the largest overall memory throughput (at 15.34 GByte/s). For the shared memory implementation, the memory throughput was highest at a window size of 32 (at 15.98 GByte/s), and dropped off when the size was larger than this, likely because it was simply doing more memory transactions across threads since less of the calculation was privatized to an individual block. We also observed that as the window size increased on the shared memory implementations, the theoretical occupancy dropped off, since each block would require more shared memory than was available on the GPU (see figure 1b). Overall, the maximum compute throughput and highest arithmetic intensity came from using the largest window size among these profiling runs (256), along with shared memory. This increase in arithmetic intensity comes at the cost of limiting the future theoretical speedup, since each block requires more shared memory and may have to be stalled by the scheduler until the space is available.

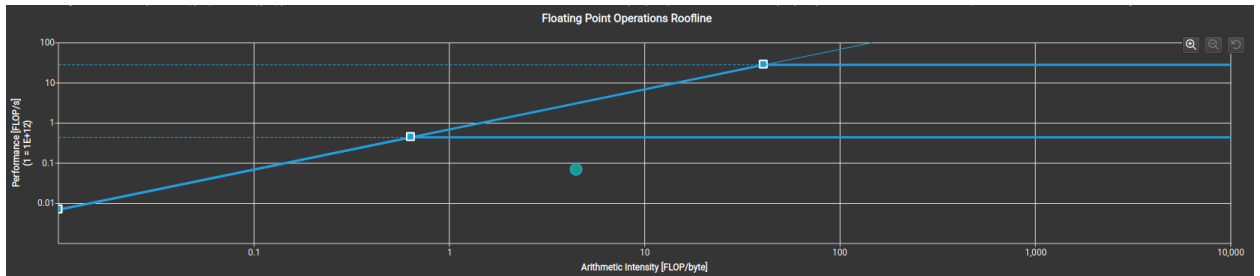


Figure 2: Roofline model for global memory windowed attention implementation, with a window size of 256. Arithmetic intensity of 4.49 FLOPs/byte.

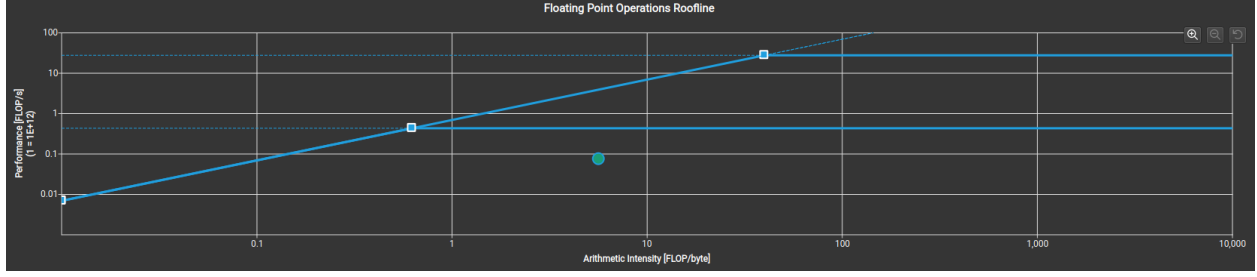


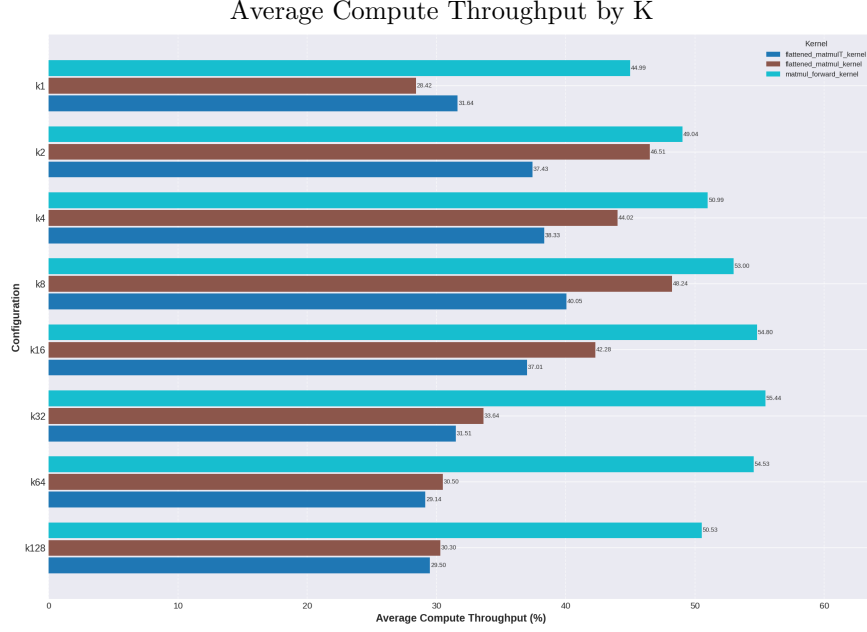
Figure 3: Roofline model for shared memory windowed attention implementation, with a window size of 256. Arithmetic intensity of 5.67 FLOPs/byte.

Overall, the relatively low memory utilization means that there is still a lot of room for improvement in our implementation - specifically, we could tweak the loads from the K , Q , and V tensors to be coalesced and loaded into a shared memory block, since our current implementation only uses shared memory to hold the windows (not the weight values from the tensors). Furthermore, the windowed attention optimizations described here were only implemented atop the baseline kernel - combined with joint register tiling to further privatize the weights during matrix multiplication on a per-thread and per-block basis, we could theoretically see an even better speedup.

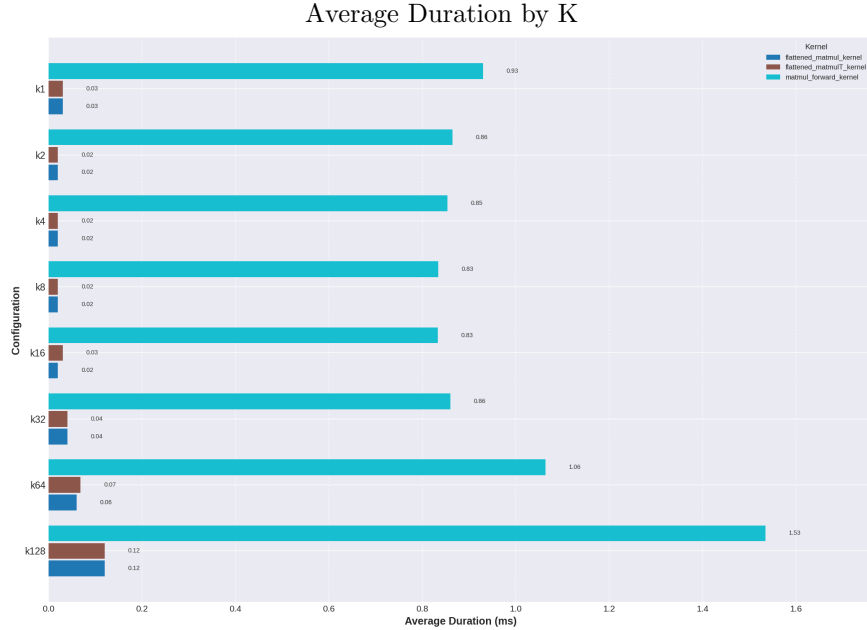
Split-K (req_9)

Split-K is a matrix multiply optimization algorithm that allows for more parallelization across the inner-dimension of the matrix multiplication. It does this by splitting the inner-dimension into K components, performing the matrix multiplication across these smaller components, and then adding the calculated values into the output. This is especially useful for skinny matrices, where the inner dimension is much larger than the outer dimensions. The downside of this approach, however, is the requirement for `atomicAdd` operations. Since many threads can be writing to the same location, atomic operations must be used to avoid race conditions.

Our implementation was applied to req_0 Joint-Register Tiled Matrix Multiply. We also did not implement reduction to reduce the number of atomic operations because we did not believe a sufficiently large K would be optimal in order for reduction to be necessary.



As K increases, the average compute throughput that each kernel is able to perform increases up to a point. For example, the matmul forward kernel has a maximum compute throughput at $K = 32$, suggesting that the increasing parallelism before that point is able to benefit performance. However, past that point, compute throughput begins to degrade. This is likely because splitting into more sections does not lead to much improvements in parallelism (e.g. each block may do 2 iterations instead of 4) but has much more expensive overhead having to launch 2x as many kernels. We see from the data that we get maximal compute throughput at $K = 32$ for the forward matmul kernel, and at $K = 8$ for the attention matmuls.



We see this same pattern when measuring durations, where the minimal duration for the matmul forward kernel occurs at $K = 8$ and $K = 16$. The exponential increase from those points onward suggests in addition to the overhead of launching the kernels, there is yet more cost due to the rising amounts of atomic operations of threads writing to the same address. This large amount of conflicts is likely what causes $K = 128$ to be much slower than the rest of the configurations.

Proposed Optimizations

In our Milestone 2 profiling report, we proposed two optimizations for Milestone 3: a Key-Value (KV) cache and multi-GPU support. After more research and discussion with course staff, we decided against implementing multi-GPU support for two primary reasons. First, we anticipated that the communication overhead between GPUs would limit the potential speedup, making it less effective compared to other optimizations that could provide similar or better performance gains with lower complexity. Second, the added complexity of multi-GPU job scheduling would complicate testing and validation on Delta. Additionally, we recognized that both a KV cache and multi-GPU support are large optimizations that require significant development effort. Given the scope of work, we chose to focus on a more approachable second optimization that would still yield performance benefits.

For this milestone, we implemented two optimizations: a KV cache and NVIDIA CUTLASS integration. The KV cache addresses the inefficiency of recomputing Key and Value matrices for all previous tokens during autoregressive generation, providing speedups for inference workloads. CUTLASS, NVIDIA’s high-performance CUDA template library for linear algebra operations, offers optimized matrix multiplication kernels that can outperform hand-tuned implementations.

Key-Value Cache

In a Transformer, each layer computes Query (Q), Key (K), and Value (V) matrices for the attention mechanism. In our baseline implementation, the model recomputes K and V for all tokens in each forward pass, even though previous tokens’ K and V values remain unchanged. A KV cache addresses this inefficiency by storing K and V matrices from previous tokens, allowing reuse across generation steps. When generating the next token, only Q for the new token is computed, while cached K and V matrices are reused for attention calculation.

We implemented a KV cache system that stores K and V matrices separately for each of the 12 transformer layers. The cache uses a layout of (L, B, NH, max_seq_len, HS) where L is the number of layers, B is batch size, NH is the number of attention heads, max_seq_len is the maximum sequence length, and HS is the head dimension. The structure allows incremental appending of the new K/V values as the tokens are processed. Our implementation supports autoregressive generation where tokens are processed sequentially (T=1), which the cache accumulating the full sequence history up to the maximum sequence length.

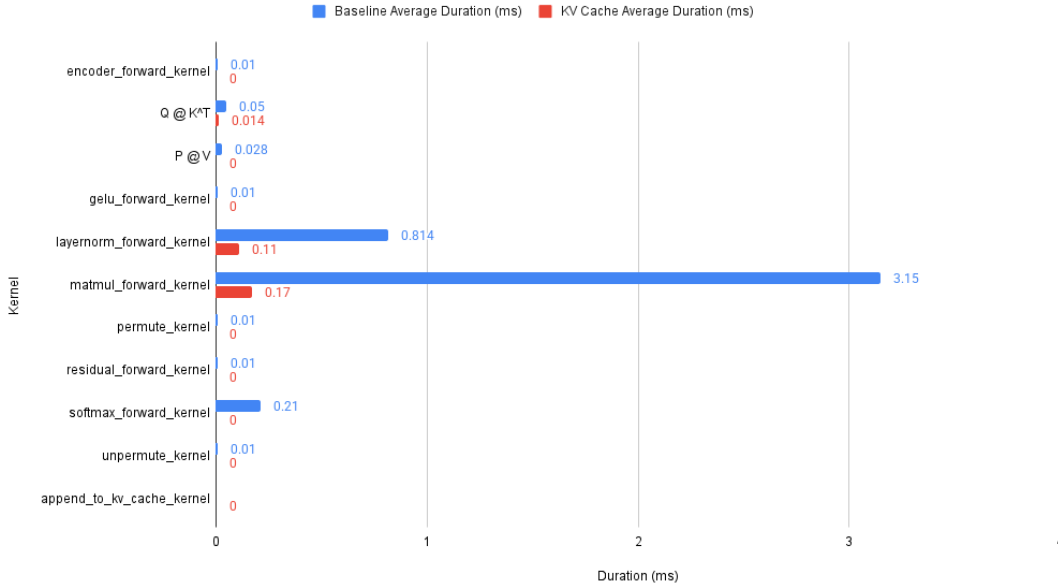
Table 5: End-to-End Generation Performance Comparison

Metric	Baseline	KV Cache	Improvement
Total Generation Time (ms)	1038.39	417.84	59.8% faster
Per-Token Latency (ms/token)	20.768	8.357	2.49× faster
Throughput (tokens/sec)	48.151	119.662	2.49× faster

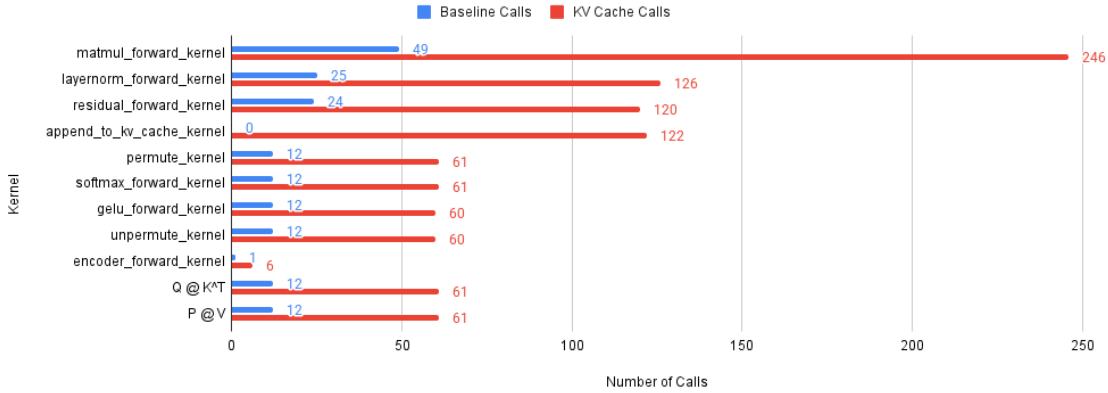
Table 6: GPU Occupancy Comparison

Metric	Baseline	KV Cache
Achieved Occupancy	71.8%	16.3%

KV Cache: Average Kernel Durations (ms)



KV Cache vs Baseline: Number of Kernel Calls



Above are the profiling results for the average kernel durations of the baseline and KV cache implementations. The KV cache implementation achieves significant per-call kernel speedups, most notably with an 18.5 $\times$ speedup for `matmul_forward_kernel`, going from 3.15ms to 0.17ms. The `layernorm_forward_kernel` had a 7.4 $\times$ speedup, going from 0.814ms to 0.11ms. The cumulative effect of these per-call improvements reduced total kernel execution time from 139.14 ms to 57.59 ms, representing a 58.6% reduction and a 2.42 $\times$ overall speedup.

These per-call speedups occur because the KV cache implementation processes tokens sequentially ($T=1$) rather than in batches ($T=100$), dramatically reducing the workload per kernel call. For the `matmul_forward_kernel`, the baseline performs matrix multiplications on 100 tokens simultaneously, while the KV cache path processes only a single token at a time. This gives a significant reduction in input size. Similarly, `layernorm_forward_kernel` in the baseline normalizes statistics across 100 tokens per call, while the KV cache version normalizes just one token, reducing both the normalization computations and reduction operations by a factor of 100. However, the speedup is not exactly 100 $\times$ because kernel launch overhead, memory access patterns, and the sequential nature of autoregressive generation introduce additional costs that partially offset the computational reduction.

The attention mechanism also changes with the KV cache. In the baseline, attention computes $Q@K^T$, where both Q and K have shape (B, NH, T, HS) , resulting in attention scores of shape (B, NH, T, T) . With the KV cache, we compute $Q(\text{new token})@K(\text{cached tokens})^T$ where Q has shape $(B, NH, 1, HS)$ and K has shape $(B, NH, \text{cache_len}, HS)$, resulting in attention scores of shape $(B, NH, 1, \text{cache_len})$. This change enables specialized kernels (`kv_cache_qk_scores` and `kv_cache_attn_out`) that are optimized for the single-token query case, further improving performance beyond the simple workload reduction.

The softmax operation also requires modification for the KV cache implementation. In the baseline, softmax applies causal masking to prevent tokens from attending to future tokens, processing only positions up to each token’s position and masking subsequent positions. However, with the KV cache, causal masking is no longer necessary because tokens are generated sequentially one at a time. When processing a new token, all previous tokens are already in the cache, so the new token should attend to all cached tokens without masking. This simplifies the computation and reduces the workload. Additionally, the baseline softmax $N \times T$ elements (one thread per query-token pair in the attention matrix), while the KV cache softmax processes only N elements (one thread per query, where each query attends to all T cached keys), further reducing computational complexity from $O(N \times T)$ to $O(N)$ per softmax operation.

While the KV cache achieves substantial per-call speedups, it also increases the frequency of kernel launches due to the autoregressive generation pattern. The total number of kernel launches increases from 171 (baseline) to 984 (KV cache), representing a $5.8\times$ increase. This pattern is consistent across most kernels: `matmul_forward_kernel` is called $5.0\times$ more frequently (49 calls in baseline versus 246 calls in KV cache), and `layernorm_forward_kernel` shows a similar $5.0\times$ increase (25 calls versus 126 calls). This increase is expected and necessary for autoregressive generation, where the baseline processes the entire input sequence ($T=100$) in a single forward pass, while the KV cache processes one token at a time ($T=1$) across approximately 100 generated tokens. Since each token must traverse all 12 transformer layers, we observe approximately $12 \text{ layers} \times 100 \text{ tokens}$ worth of kernel launches, which aligns with our measured 984 total launches after accounting for initialization overhead.

Despite the increased launch frequency, the KV cache achieves net speedup because the per-call speedups ($7.4\times$ to $18.5\times$) compensate for the $5\times$ increase in launch count. For example, while `matmul_forward_kernel` is called $5.0\times$ more frequently, each call executes $18.5\times$ faster, reducing total execution time from approximately 154 ms ($49 \text{ calls} \times 3.15 \text{ ms}$) to 42 ms ($246 \text{ calls} \times 0.17 \text{ ms}$), representing a 73% reduction. Similarly, `layernorm_forward_kernel` total time decreases from 20.4 ms ($25 \text{ calls} \times 0.814 \text{ ms}$) to 13.9 ms ($126 \text{ calls} \times 0.11 \text{ ms}$), a 32% reduction. Cumulatively, the total duration across all kernels decreases from 139.14 ms (baseline) to 57.59 ms (KV cache), representing a 58.6% reduction and a $2.42\times$ overall speedup. The increased kernel launch overhead from more frequent launches is negligible compared to the computational time saved per kernel execution, as evidenced by these overall runtime improvements.

The KV cache trades occupancy for computation efficiency. While achieved occupancy decreases from 71.8% (baseline) to 16.3% (KV cache), this reduction is expected and acceptable given the smaller per-token workloads. With $T=1$, each kernel processes significantly fewer elements, resulting in fewer active threads and lower occupancy. However, the per-call speedups and reduced total execution time demonstrate that computational efficiency outweighs the reduced resource utilization. The lower occupancy reflects the autoregressive generation workload, where sequential processing of small workloads is more efficient than batch processing of large workloads, even if it underutilizes GPU resources. This trade-off is beneficial for inference workloads where latency per token is more important than maximizing GPU utilization.

CUTLASS

CUTLASS (CUDA Templates for Linear Algebra Subroutines) is NVIDIA’s open-source C++ template library for implementing high-performance CUDA kernels targeting linear algebra operations, particularly matrix multiplication (GEMM). Unlike cuBLAS, which provides pre-compiled, optimized library functions, CUTLASS is a header-only template library that generates optimized kernels at compile time. This design

allows developers to customize kernel implementations for specific use cases while leveraging NVIDIA’s extensive optimization research, including advanced techniques such as tensor core utilization, sophisticated memory access patterns, register tiling, shared memory management, and kernel fusion strategies.

In Milestone 2, we implemented cuBLAS for matrix operations, which showed strong performance through NVIDIA’s highly optimized, pre-compiled kernels. However, cuBLAS is a black-box library that offers limited customization and insight into the underlying optimization techniques. CUTLASS, by contrast, provides source-level access to state-of-the-art GEMM implementations, allowing us to understand and potentially customize the optimization strategies. This comparison is valuable because it offers flexibility, transparency, and the potential for application-specific optimizations.

0.0.1 Implementation

Our CUTLASS implementation handles multi-head attention by launching separate GEMM kernels for each (batch, head) combination. Specifically, the attention matrix multiplications iterate over $\text{BNH} = \text{Batch} \times \text{NumHeads}$ dimensions:

```
for (int bnh = 0; bnh < BNH; ++bnh) {
    // Compute pointer offsets for this head
    CutlassGemm gemm_op;
    gemm_op(); // Launch kernel for this specific head
}
```

With a batch size of 4 and 12 attention heads, this results in 48 separate CUTLASS kernel launches for each of the two attention matrix multiplications per layer. Based on the profiling results we will discuss below, this incurs a large overhead that the kernel speedup might not outweigh. In the future, we would try CUTLASS’ batched GEMM operations that can process multiple independent matrix multiplications in a single kernel launch. This would reduce the kernel launch count and potentially lead to more speedup.

0.0.2 Initial CUTLASS Analysis

When setting up CUTLASS, we built a parameter structure that specifies the location in the K , Q , and V tensors based on the combined batch size and number of heads index and calls the GEMM kernel in a sequential loop. With a batch size of 4 and 12 heads, this resulted in a total of 48 calls to the CUTLASS kernel. The repeated sequential calls to this kernel also negatively impacted the cache hit rates, with L1

Table 7

Optimization	Matmul Duration ($Q * K^T$)	Compute Throughput	Memory Throughput
CUTLASS	0.779 ms	0.66% (average)	1.5% (average)
Baseline	0.005 ms	29.03%	85.41%

cache hit rates lowering to around 50%, and L2 lowering to 69%, down from 91% and 88% respectively in the baseline $Q * K^T$ matrix multiplication kernel.

0.0.3 Comparison with cuBLAS

Although less flexible than CUTLASS, cuBLAS’s actual performance far exceeds what we were able to achieve with CUTLASS in this approach. Since cuBLAS optimizes the grid and block parameters automatically across the entire tensor, we were able to get significantly better runtimes (0.0016 ms) and much better memory utilization (10x memory bandwidth increase, with an L1 cache hit rate of 91% and L2 of 88%). With additional tweaking, we might have been able to get better performance from CUTLASS, but as it stands the black-box cuBLAS kernels were faster and easier to work with when dealing with GEMM in attention and the feedforward kernels.

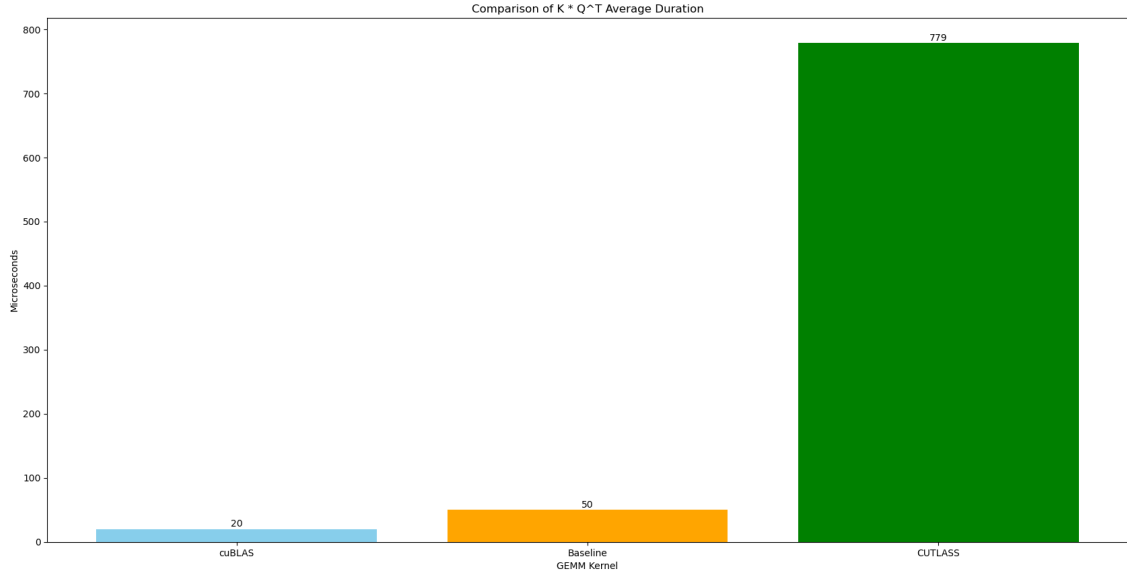


Figure 4: Average Duration of $Q * K^T$ matrix multiplication. CUTLASS is much slower than our baseline or the (fastest) cuBLAS implementation. Since CUTLASS consists of GEMM kernels for each tensor slice, the individual kernel durations were averaged and scaled by 48x to approximate launching a single kernel across the entire B, NH dimension.

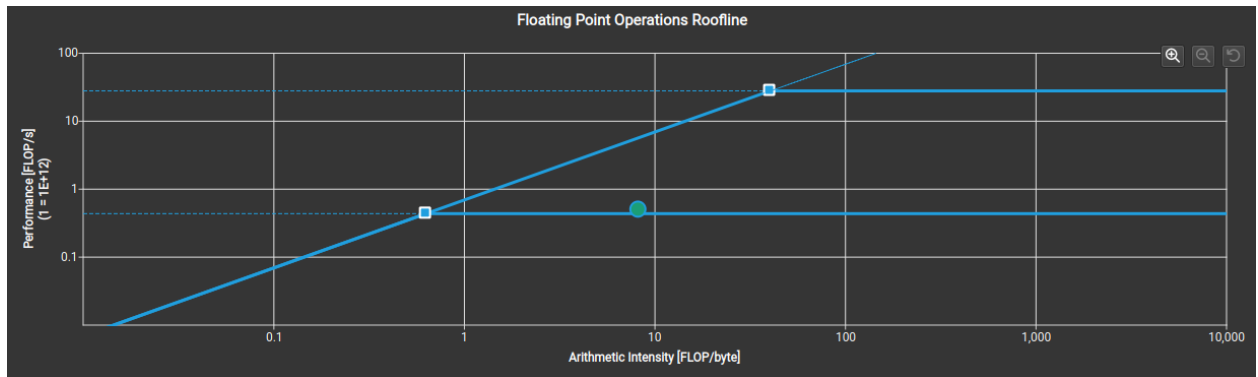


Figure 5: Roofline graph using cuBLAS GEMM kernel

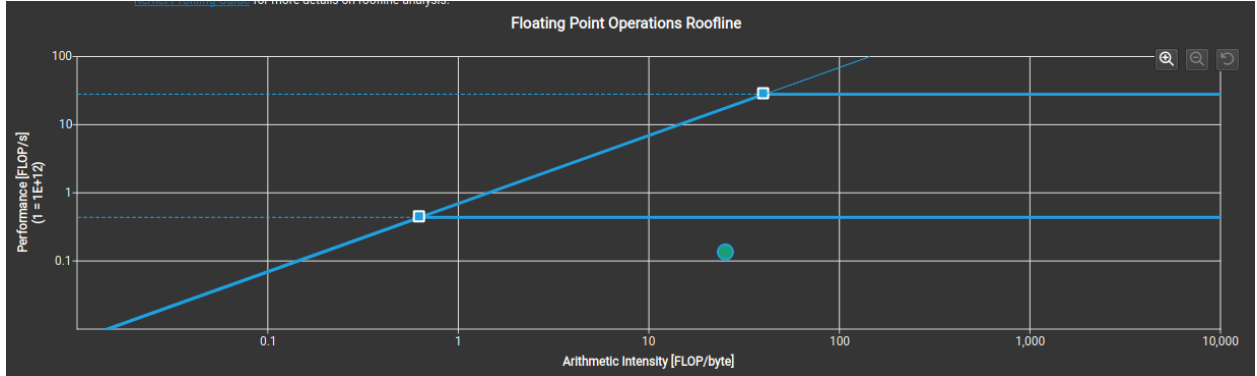


Figure 6: Roofline graph using CUTLASS GEMM kernel, for one slice of the total K, Q tensors (based on $b * head_{idx}$ index calculation).

Conclusion and Interesting Findings

Altogether, using the KV-cache to avoid recomputation presented the single largest speedup among our optimizations.

For a final, superoptimized version of our kernel implementations, we tried using the cuBLAS implementation of the matrix multiplication kernels called within `attention_forward` and `matmul_forward` during the standard path for KV-cache (the standard attention block/what is called if the value is not cached). This actually ended up being considerably slower than the default KV-cache implementation - our processing rate decreased to 26.165 tokens/sec, and the total inference time increased to 1910 ms (down from 119 tokens/sec and up from 417 ms in the standard KV-cache implementation, which leveraged our own matrix multiplication kernels).

We were unable to complete a full profiling run of this particular kernel configuration, but we suspect that this slowdown comes from the fact that tokens are processed sequentially, and so the actual occupancy of the cuBLAS matrix multiplication kernel was considerably lower than the theoretical upper bound. Another attempt to merge our KV-cache and tensor core operations gave back similar results - our total inference time was 1023 ms, and our throughput was still considerably lower than KV-cache alone at 48.843 tokens/sec. Our best result was a combination of the KV-cache with flash attention, but the impact on total inference time seems within the range of standard times we observed with KV-cache alone.

Table 8

KV-Cache with:	Token generation time (ms)	Token throughput (tokens/sec)
nothing else	417	119
cuBLAS	1910	26.165
Tensor cores	1023	48.843
Flash attention	399	125.036
Joint register tiling	921	54.237
CUTLASS	624	80.818

If we were to continue combining optimizations, we would have liked to apply our configuration sweep script (from req_5) to each of these combined optimizations. In particular, further optimizing flash attention (for instance, using FlashAttentionV2) combined with the KV-cache, reduction, and cuBLAS functions could yield even better speedups than what we were able to achieve here.